Practical 1

Introduction to DevOps Tools and Practices.
a. Research and document the importance of DevOps in modern software development.
b. Install and explore one DevOps tool (e.g., Jenkins, Docker, or Ansible).
c. Create a diagram showing how CI/CD, monitoring, and Infrastructure as Code work together

Introduction to DevOps Tools and Practices

A. Importance of DevOps in Modern Software Development

What is DevOps?

DevOps is a software development approach that combines **Development (Dev)** and **Operations (Ops)** teams to improve collaboration, automate workflows, and deliver software faster and more reliably.

Importance of DevOps

1. **Faster Software Delivery**
    - Automates building, testing, and deployment processes.
    - Reduces time-to-market for new features and updates.
2. **Improved Collaboration**
    - Developers and operations teams work together throughout the software lifecycle.
    - Reduces communication gaps and misunderstandings.
3. **Higher Software Quality**
    - Continuous testing identifies bugs early.
    - Automated quality checks improve reliability.
4. **Increased Deployment Frequency**
    - Organizations can release updates multiple times a day instead of monthly or quarterly.
5. **Better System Reliability**
    - Continuous monitoring helps detect and fix issues quickly.
    - Infrastructure automation reduces configuration errors.
6. **Scalability and Flexibility**
    - Cloud platforms and Infrastructure as Code (IaC) make scaling easier.

Common DevOps Practices

- Continuous Integration (CI)

- Continuous Delivery/Deployment (CD)
- Infrastructure as Code (IaC)
- Monitoring and Logging
- Automated Testing
- Version Control (Git)
- Containerization

Popular DevOps Tools

Category	**Tools**
Version Control	Git, GitHub, GitLab
CI/CD	Jenkins, GitHub Actions, GitLab CI
Containers	Docker, Podman
Container Orchestration	Kubernetes
Configuration Management	Ansible, Puppet, Chef
Monitoring	Prometheus, Grafana, Nagios

B. Installation and Exploration of Docker

Why Docker?

Docker is a containerization platform that packages applications and their dependencies into lightweight containers, ensuring consistent behavior across environments.

Installation Steps (Windows)

1. Visit the official Docker website:
    - Docker Desktop
2. Download Docker Desktop.
3. Run the installer and follow the setup wizard.
4. Restart the system if required.
5. Verify installation:

docker --version

Expected output:

Docker version XX.X.X

Basic Docker Commands

Check Docker Status

docker info

Pull an Image

docker pull nginx

View Images

docker images

Run a Container

docker run -d -p 8080:80 nginx

List Running Containers

docker ps

Stop a Container

docker stop <container-id>

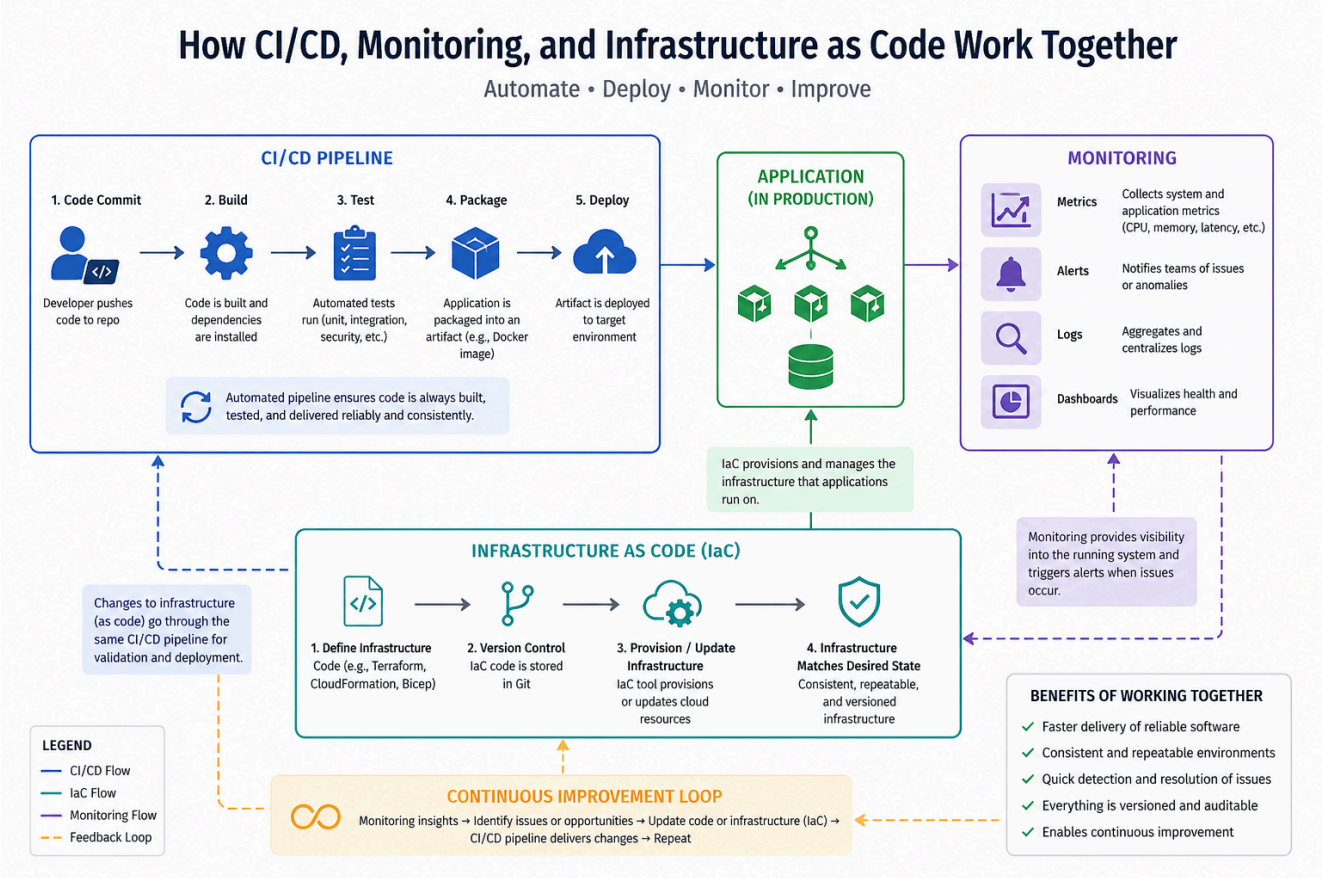
Observations

- Docker containers start quickly.
- Applications run consistently across systems.
- Resource usage is lower compared to virtual machines.
- Simplifies deployment and scaling.

Advantages of Docker

- Portability
- Isolation
- Faster deployments
- Reduced infrastructure costs
- Supports microservices architecture

C. Diagram: How CI/CD, Monitoring, and Infrastructure as Code Work Together



Explanation of the Diagram

1. Developers commit code to a Git repository.
2. The CI pipeline automatically builds and tests the code.
3. If tests pass, the CD pipeline deploys the application.
4. Infrastructure as Code tools provision and configure servers automatically.
5. The application runs in production.
6. Monitoring tools collect performance metrics and logs.
7. Alerts and feedback help teams identify issues and improve future releases.

Conclusion

DevOps enables organizations to deliver software faster, more reliably, and with higher quality. Tools such as Docker, Jenkins, Ansible, Prometheus, and Grafana support automation throughout the software lifecycle. By integrating CI/CD pipelines, Infrastructure as Code, and monitoring systems, teams can achieve continuous improvement and efficient software delivery.

Practical 2:

Aim:
Building a REST API with Flask or FastAPI.
a. Create a simple REST API using Flask or FastAPI with CRUD operations for a "To-Do List" application.
b. Test the API using Postman or curl.

Solution:

Step 1: Check if Python is Installed

Open **Command Prompt (CMD)** and run:

python --version

output:
Python 3.12.4

Step 2: Create a Project Folder

Create a folder for your Flask project:

Flask_Project
Open Command Prompt and navigate to it:

cd path\to\Flask_Project
Example:

cd C:\Users\Nikhita\Documents\Flask_Project

Step 3: Create a Virtual Environment (Recommended)

Inside the project folder, run:

python -m venv venv
This creates a virtual environment named venv.

Activate it:

Windows
venv\Scripts\activate
You should see:

(venv) C:\Users\Nikhita\Documents\Flask_Project>

Step 4: Install Flask
With the virtual environment activated, run:

pip install flask

Step 5: Verify Installation

pip show flask

or

python -m flask --version

Example output:

Flask 3.x.x
Werkzeug 3.x.x

Step 6: Create Your First Flask App

Create a file named app.py:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Hello, Flask!"

if __name__ == '__main__':
    app.run(debug=True)
```

Step 7: Run the Flask App

In Command Prompt:

python app.py

You should see:

* Running on http://127.0.0.1:5000
Open your browser and visit:

http://127.0.0.1:5000
Output:

Hello, Flask!

Code 2:

Replace the code with:

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return jsonify({
        "message": "Welcome to my first Flask API",
        "status": "success"
    })

if __name__ == '__main__':
    app.run(debug=True)
```

Run again:

```
python app.py
```

Visit:

```
http://127.0.0.1:5000
```

Output:

```
{
  "message": "Welcome to my first Flask API",
  "status": "success"
}
```

This is your **first REST API endpoint** in Flask.

Code 3:

```
from flask import Flask, jsonify, request
```

```
app = Flask(__name__)

# Sample data storage
todos = [
    {
        "id": 1,
        "title": "Learn Flask",
        "completed": False
    }
]

# CREATE a new task
@app.route('/todos', methods=['POST'])
def create_todo():
    data = request.get_json()

    new_todo = {
        "id": len(todos) + 1,
        "title": data['title'],
        "completed": False
    }

    todos.append(new_todo)

    return jsonify(new_todo), 201


# READ all tasks
@app.route('/todos', methods=['GET'])
def get_todos():
    return jsonify(todos)


# READ a single task
@app.route('/todos/<int:id>', methods=['GET'])
def get_todo(id):
    todo = next((t for t in todos if t['id'] == id), None)

    if todo is None:
        return jsonify({"message": "Task not found"}), 404

    return jsonify(todo)


# UPDATE a task
@app.route('/todos/<int:id>', methods=['PUT'])
def update_todo(id):
    todo = next((t for t in todos if t['id'] == id), None)

    if todo is None:
        return jsonify({"message": "Task not found"}), 404

    data = request.get_json()

    todo['title'] = data.get('title', todo['title'])
    todo['completed'] = data.get('completed', todo['completed'])
```

```
    return jsonify(todo)


# DELETE a task
@app.route('/todos/<int:id>', methods=['DELETE'])
def delete_todo(id):
    global todos

    todo = next((t for t in todos if t['id'] == id), None)

    if todo is None:
        return jsonify({"message": "Task not found"}), 404

    todos = [t for t in todos if t['id'] != id]

    return jsonify({"message": "Task deleted successfully"})

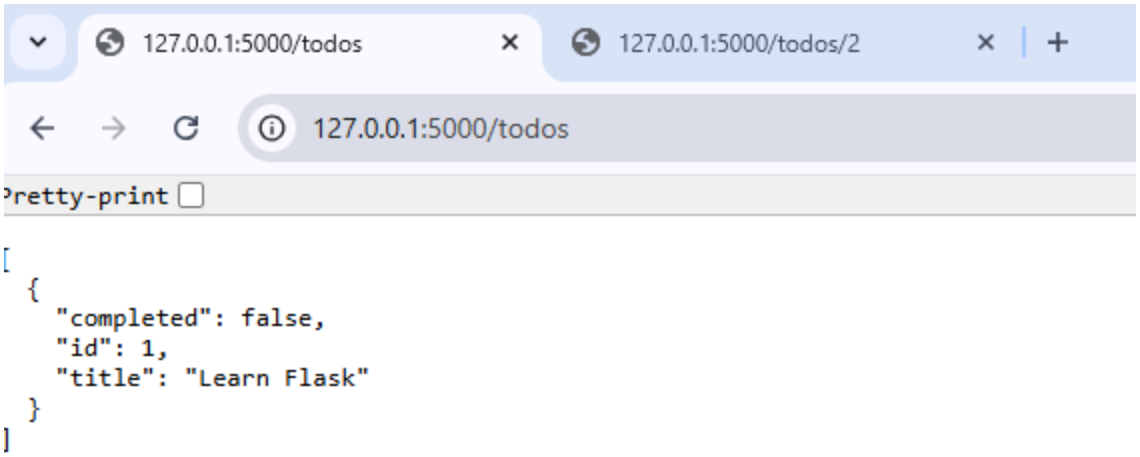

if __name__ == '__main__':
    app.run(debug=True)
```

Run again:

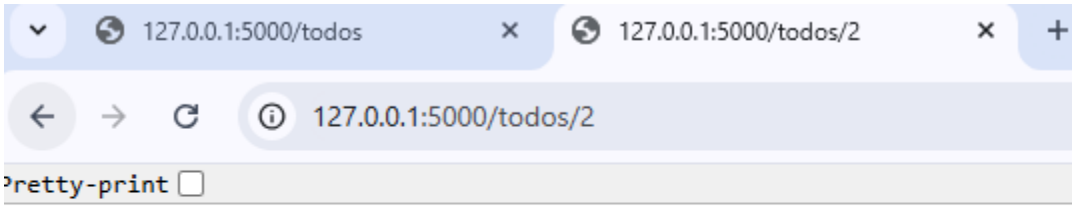
```
python app.py
```

Visit:

http://127.0.0.1:5000/todos
http://127.0.0.1:5000/todos/2




```
[
  {
    "completed": false,
    "id": 1,
    "title": "Learn Flask"
  }
]
```


```
{
  "message": "Task not found"
}
```

```
C:\Users\192>curl http://127.0.0.1:5000/todos
[
  {
    "completed": false,
    "id": 1,
    "title": "Learn Flask"
  }
]

C:\Users\192>curl http://127.0.0.1:5000/todos/1
{
  "completed": false,
  "id": 1,
  "title": "Learn Flask"
}
```

```
C:\Users\192>curl -X DELETE http://127.0.0.1:5000/todos/2
{
  "message": "Task not found"
}
```

```
C:\Users\192>curl -X POST http://127.0.0.1:5000/todos -H "Content-Type: applic
ation/json" -d "{\"title\":\"Complete REST API Assignment\"}"
{
  "completed": false,
  "id": 2,
  "title": "Complete REST API Assignment"
}
```

1. GET All Tasks

```
curl http://127.0.0.1:5000/todos
```

Output:

```
[
  {
    "id": 1,
    "title": "Learn Flask",
    "completed": false
  }
]
```

---

2. POST (Create)

```
curl -X POST http://127.0.0.1:5000/todos -H "Content-Type: application/json" -d
"{\"title\":\"Complete REST API Assignment\"}"
```

Output:

```
{
  "id": 2,
  "title": "Complete REST API Assignment",
  "completed": false
}
```

---

3. PUT (Update)

```
curl -X PUT http://127.0.0.1:5000/todos/1 -H "Content-Type: application/json"
-d "{\"title\":\"Learn Flask API\",\"completed\":true}"
```

Output:

```
{
  "id": 1,
  "title": "Learn Flask API",
  "completed": true
}
```

---

4. DELETE

```
curl -X DELETE http://127.0.0.1:5000/todos/1
```

Output:

```
{
  "message": "Task deleted successfully"
}
```

---

5. Verify Deletion

```
curl http://127.0.0.1:5000/todos
```

Output:

```
[
  {
    "id": 2,
    "title": "Complete REST API Assignment",
    "completed": false
  }
]
```

Quick Reference for Viva

HTTP Method	Endpoint	Purpose
GET	`/todos`	Retrieve all tasks
GET	`/todos/1`	Retrieve a specific task
POST	`/todos`	Create a new task
PUT	`/todos/1`	Update an existing task
DELETE	`/todos/1`	Delete a task

This summary matches the Flask code you provided and is sufficient for demonstrating all CRUD operations using `curl`.

Practical: Git Repository Management

Aim

Create a Git repository, initialize it, and add a Python project.

Software Required

- Git for Windows (already installed)
- Visual Studio Code
- Python (installed)
- GitHub account (optional for Part A)

Folder Structure

You will create this folder:

```
Git_Practical
├── hello.py
├── README.md
└── .git
```

Step 1: Open Visual Studio Code

1. Click **Start Menu**.
2. Search for **Visual Studio Code**.
3. Open it.

Step 2: Open the Terminal

In VS Code,

Click

Terminal

↓

New Terminal

A terminal opens at the bottom.

It looks like

PS C:\Users\YourName>

(PS means PowerShell.)

Step 3: Go to Desktop

Type

cd Desktop

Press **Enter**

Now you are inside the Desktop folder.

Example

PS C:\Users\Nikhita\Desktop>

Step 4: Create a Folder

Type

mkdir Git_Practical

Press Enter.

Nothing will happen on the screen.

It simply creates the folder.

Step 5: Move into the Folder

Type

cd Git_Practical

Press Enter.

Now terminal becomes

PS C:\Users\Nikhita\Desktop\Git_Practical>

Step 6: Open this Folder in VS Code

Type

code .

Press Enter.

The folder opens inside VS Code.

You should now see

Git_Practical

on the left Explorer.

Step 7: Create Python File

Click

New File

Name it

hello.py

Step 8: Write Python Program

Inside **hello.py**

print("Hello Git!")

Save

Ctrl + S

Step 9: Create README File

Again click

New File

Name it

README.md

Write

Git Practical

This is my first Git repository.

Project Name: Python Demo

Created by: Your Name

Save it.

Step 10: Check Folder

Your folder now contains

Git_Practical

hello.py

README.md

Step 11: Initialize Git Repository

Go to Terminal.

Type

git init

Press Enter.

Output

Initialized empty Git repository in
C:/Users/Nikhita/Desktop/Git_Practical/.git/

Now Git creates a hidden folder

.git

This folder stores all version history.

Step 12: Check Git Status

Type

git status

Output

On branch master

No commits yet

Untracked files:

hello.py
README.md

Meaning

Git has found two files.

But they are **not being tracked**.

Step 13: Add Files

To add all files

git add .

Press Enter.

OR

Add only one file

git add hello.py

Step 14: Check Status Again

Type

git status

Output

Changes to be committed:

new file: hello.py

new file: README.md

Now Git is ready to save these files.

Step 15: Configure Git (First Time Only)

If this is your first Git project, configure your identity.

Type

git config --global user.name "Your Name"

Example

git config --global user.name "Nikhita"

Press Enter.

Now email

git config --global user.email "your@email.com"

Example

git config --global user.email "nikhita@gmail.com"

Press Enter.

Step 16: Commit the Project

Now save the project permanently.

Type

git commit -m "Initial Python Project"

Press Enter.

Example output

[master (root-commit) abc1234]

2 files changed

create mode 100644 hello.py

create mode 100644 README.md

Congratulations!

Your first Git commit is complete.

Step 17: Check Status

Type

git status

Output

On branch master

nothing to commit, working tree clean

This means:

- ✅ All changes are saved.
- ✅ Repository is up to date.
- ✅ There are no pending changes.

Step 18: View Commit History

Type

git log

Output

commit a12bc34d56...

Author: Nikhita

Date: ...

Initial Python Project

This shows the commit history.

Step 19: Modify the Python File

Open

hello.py

Change it to

```
print("Hello Git!")
print("Welcome to Git Repository")
```

Save the file.

Step 20: Check Status

Type

git status

Output

modified: hello.py

Git detects that the file has changed.

Step 21: Save the Changes

Stage the modified file:

git add hello.py

Commit the changes:

git commit -m "Updated hello.py"

Step 22: View All Commits

Type

git log

Now you'll see two commits:

Updated hello.py

Initial Python Project

Final Folder Structure

Git_Practical

```
├── .git
├── hello.py
└── README.md
```

Commands Summary

Step	Command	Purpose
1	cd Desktop	Go to Desktop
2	mkdir Git_Practical	Create project folder
3	cd Git_Practical	Open the folder
4	code .	Open folder in VS Code
5	git init	Initialize Git repository

Step	Command	Purpose
6	git status	Check repository status
7	git add .	Stage all files
8	git commit -m "Initial Python Project"	Save the first version
9	git log	View commit history
10	git add hello.py	Stage modified file
11	git commit -m "Updated hello.py"	Save changes
12	git status	Verify working tree is clean

Viva Questions (with Answers)

Q1. What is Git?
Answer: Git is a distributed version control system used to track changes in files and manage source code during software development.

Q2. What is a Git repository?
Answer: A Git repository is a folder that contains project files along with a hidden .git directory where Git stores version history and metadata.

Q3. What does git init do?
Answer: It initializes a new Git repository by creating a hidden .git folder in the project directory.

Q4. What is the purpose of git add?
Answer: It stages new or modified files so they are ready to be committed.

Q5. What does git commit do?
Answer: It saves a snapshot of the staged changes in the repository with a descriptive message.

This completes **Part (a): Create a Git repository, initialize it, and add a Python project** in a detailed, beginner-friendly manner. Once you're comfortable with this, you can proceed to **Part (b): Branching, merging, and pull requests**.

1. Create a new branch.
2. Add new code in the branch.
3. Commit the changes.
4. Push the branch to GitHub.
5. Create a Pull Request (PR).
6. Merge the Pull Request into the `main` branch.
7. Pull the updated `main` branch.

Important: A **Pull Request (PR)** is a feature of **GitHub**, not Git itself. So, to demonstrate a PR, you'll need a GitHub account and a remote repository.

Practical B: Demonstrate Branching and Merging Using Pull Requests

Prerequisites

Before starting, make sure you have:

- ✅ Git installed
- ✅ VS Code installed
- ✅ GitHub account
- ✅ Completed Part (a)
- ✅ Your project (`Git_Practical`) already committed

Current Project Structure

```
Git_Practical
│
├── hello.py
├── README.md
└── .git
```

Step 1: Open Your Project

Open **VS Code**.

Go to:

File → Open Folder → Git_Practical

Open the terminal:

Terminal → New Terminal

Step 2: Check the Current Branch

Type:

```
git branch
```

Output:

```
* master
```

or

```
* main
```

If your repository shows **master**, use `master` in the commands below instead of `main`.

Step 3: Rename `master` to `main` (Optional)

Modern repositories use `main`.

If your branch is `master`, type:

```
git branch -M main
```

Check again:

```
git branch
```

Output:

```
* main
```

Step 4: Create a New Feature Branch

Create a branch called **feature-login**:

```
git checkout -b feature-login
```

Output:

```
Switched to a new branch 'feature-login'
```

Step 5: Verify the Branch

```
git branch
```

Output:

```
* feature-login
main
```

The `*` indicates the current branch.

Step 6: Modify the Python File

Open:

```
hello.py
```

Current code:

```
print("Hello Git!")
print("Welcome to Git Repository")
```

Add new lines:

```
print("Hello Git!")
print("Welcome to Git Repository")

print("Login Feature Added")
print("Feature Branch Example")
```

Save the file (**Ctrl + S**).

Step 7: Check Status

```
git status
```

Output:

```
modified: hello.py
```

Step 8: Stage the Changes

```
git add hello.py
```

Step 9: Commit the Changes

```
git commit -m "Added login feature"
```

Example output:

```
[feature-login abc123]

1 file changed
```

Step 10: View Commit History

```
git log --oneline
```

Example:

```
7b8c91 Added login feature
2a3bc4 Updated hello.py
1f23de Initial Python Project
```

Step 11: Create a Repository on GitHub

1. Open GitHub.
2. Click **New Repository**.
3. Repository name:

```
Git_Practical
```

4. Keep it **Public**.
5. Do **not** add a README (you already have one).
6. Click **Create Repository**.

Step 12: Connect Local Repository to GitHub

Copy your GitHub repository URL, for example:

```
https://github.com/yourusername/Git_Practical.git
```

Run:

```
git remote add origin https://github.com/yourusername/Git_Practical.git
```

Verify:

```
git remote -v
```

Output:

```
origin https://github.com/yourusername/Git_Practical.git
```

Step 13: Push the Main Branch

Switch to the main branch:

```
git checkout main
```

Push it:

```
git push -u origin main
```

Step 14: Push the Feature Branch

Switch back:

```
git checkout feature-login
```

Push:

```
git push -u origin feature-login
```

Output:

```
Branch 'feature-login' set up to track remote branch.
```

Step 15: Create a Pull Request

Go to GitHub.

You will see:

```
Compare & Pull Request
```

Click it.

Step 16: Fill Pull Request Details

Title:

```
Added Login Feature
```

Description:

```
This pull request adds a login feature to the Python project.
```

Click:

```
Create Pull Request
```

Step 17: Merge the Pull Request

Click:

```
Merge Pull Request
```

Then:

```
Confirm Merge
```

GitHub will display:

```
Pull Request Successfully Merged
```

Step 18: Switch Back to Main

In VS Code:

```
git checkout main
```

Step 19: Pull the Latest Changes

```
git pull origin main
```

Output:

```
Updating...
Fast-forward
```

Step 20: Verify the Merge

Open:

```
hello.py
```

You should see:

```
print("Hello Git!")
print("Welcome to Git Repository")

print("Login Feature Added")
print("Feature Branch Example")
```

The feature branch changes are now in the `main` branch.

Step 21: View Branches

```
git branch
```

Output:

```
feature-login
* main
```

Step 22: Delete the Feature Branch (Optional)

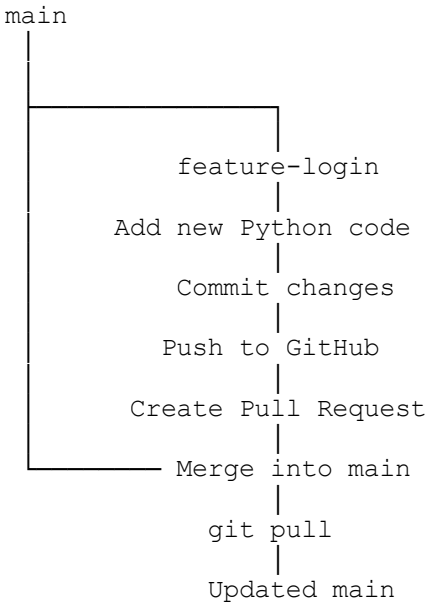
Delete the local branch:

```
git branch -d feature-login
```

Delete the remote branch:

```
git push origin --delete feature-login
```

Workflow Diagram



Git Commands Used

Command	Purpose
`git branch`	Display branches
`git checkout -b feature-login`	Create and switch to a new branch
`git add hello.py`	Stage the modified file
`git commit -m "Added login feature"`	Commit the changes
`git remote add origin <repository-url>`	Connect local repository to GitHub
`git push -u origin main`	Push the main branch
`git push -u origin feature-login`	Push the feature branch

Command	Purpose
`git checkout main`	Switch to the main branch
`git pull origin main`	Download merged changes
`git branch -d feature-login`	Delete the local feature branch

Viva Questions

Q1. What is a Git branch?
Answer: A Git branch is a separate line of development that allows developers to work on new features or bug fixes without affecting the main code.

Q2. Why is branching used?
Answer: Branching allows developers to develop and test features independently before merging them into the main project.

Q3. What is a Pull Request (PR)?
Answer: A Pull Request is a GitHub feature used to propose changes from one branch to another. It allows code review, discussion, and approval before merging.

Q4. What is the purpose of merging?
Answer: Merging combines changes from one branch into another, usually from a feature branch into the `main` branch.

Q5. What is the difference between Git and GitHub?
Answer: Git is a version control system used locally to track changes, while GitHub is a cloud-based platform that hosts Git repositories and provides collaboration features such as Pull Requests.

This completes **Part (b): Demonstrate branching by creating a new feature branch, adding code, and merging it back into the `main` branch using Pull Requests** in a detailed, beginner-friendly manner.

Step 11.1: Sign in to GitHub

1. Open your web browser (Chrome, Edge, Firefox, etc.).
2. Go to **https://github.com**.
3. Enter your **username/email** and **password**.
4. Click **Sign in**.

Absolutely! This is the **third Git practical** and is commonly asked in FYIT practical exams.

Practical (c): Simulate a Merge Conflict and Resolve It

Aim

To create a merge conflict between two branches and resolve it successfully.

Prerequisites

You should have completed:

- ✅ Part (a): Created a Git repository
- ✅ Part (b): Created a feature branch and merged it

Your project should look like:

```
Git_Practical
│
├── hello.py
├── README.md
└── .git
```

Current `hello.py`:

```
print("Hello Git!")
print("Welcome to Git Repository")
```

Step 1: Open the Project

Open **VS Code**.

Open the project folder:

```
Git_Practical
```

Open the terminal:

```
Terminal → New Terminal
```

Step 2: Check the Current Branch

Type:

```
git branch
```

Output:

```
* main
```

You should be on the `main` branch.

Step 3: Create a New Branch

Create a branch named **feature-conflict**.

```
git checkout -b feature-conflict
```

Output:

```
Switched to a new branch 'feature-conflict'
```

Step 4: Modify `hello.py` in the Feature Branch

Open:

```
hello.py
```

Current code:

```
print("Hello Git!")
print("Welcome to Git Repository")
```

Replace it with:

```
print("Hello Git!")
print("Feature Branch Version")
```

Save (**Ctrl + S**).

Step 5: Commit the Changes

Check status:

```
git status
```

Stage the file:

```
git add hello.py
```

Commit:

```
git commit -m "Modified hello.py in feature branch"
```

Output:

```
1 file changed
```

Step 6: Switch Back to Main

```
git checkout main
```

Output:

```
Switched to branch 'main'
```

Step 7: Modify the Same Line in Main

Open:

```
hello.py
```

Current version (on `main`):

```
print("Hello Git!")
print("Welcome to Git Repository")
```

Replace it with:

```
print("Hello Git!")
print("Main Branch Version")
```

Save the file.

Step 8: Commit the Main Branch Changes

```
git add hello.py
```

Commit:

```
git commit -m "Modified hello.py in main branch"
```

Step 9: Merge the Feature Branch

Now merge:

```
git merge feature-conflict
```

Git will display:

```
Auto-merging hello.py

CONFLICT (content): Merge conflict in hello.py

Automatic merge failed; fix conflicts and then commit the result.
```

🎉 **Congratulations! You have successfully created a merge conflict.**

Step 10: Open the Conflicted File

Open:

```
hello.py
```

Git automatically inserts conflict markers.

You will see:

```
print("Hello Git!")

<<<<<<< HEAD
print("Main Branch Version")
=======
print("Feature Branch Version")
>>>>>>> feature-conflict
```

What Do These Symbols Mean?

`<<<<<<< HEAD`

Current branch (`main`)

`=======`

Separator

`>>>>>>> feature-conflict`

Code from the `feature-conflict` branch.

Step 11: Resolve the Conflict

Decide what you want to keep.

Option 1 (Keep Main)

```
print("Hello Git!")
print("Main Branch Version")
```

Option 2 (Keep Feature)

```
print("Hello Git!")
print("Feature Branch Version")
```

Option 3 (Recommended for Practical)

Keep **both**:

```
print("Hello Git!")
print("Main Branch Version")
print("Feature Branch Version")
```

Delete **all conflict markers** (<<<<<<<, =======, >>>>>>>).

Save the file.

Step 12: Stage the Resolved File

```
git add hello.py
```

Step 13: Complete the Merge

Commit the resolved merge:

```
git commit -m "Resolved merge conflict"
```

Output:

```
[main]
Resolved merge conflict
```

Step 14: Verify the Merge

Type:

```
git log --oneline
```

Example:

```
abc123 Resolved merge conflict
456xyz Modified hello.py in main branch
789def Modified hello.py in feature branch
```

Step 15: Check Repository Status

```
git status
```

Output:

```
On branch main

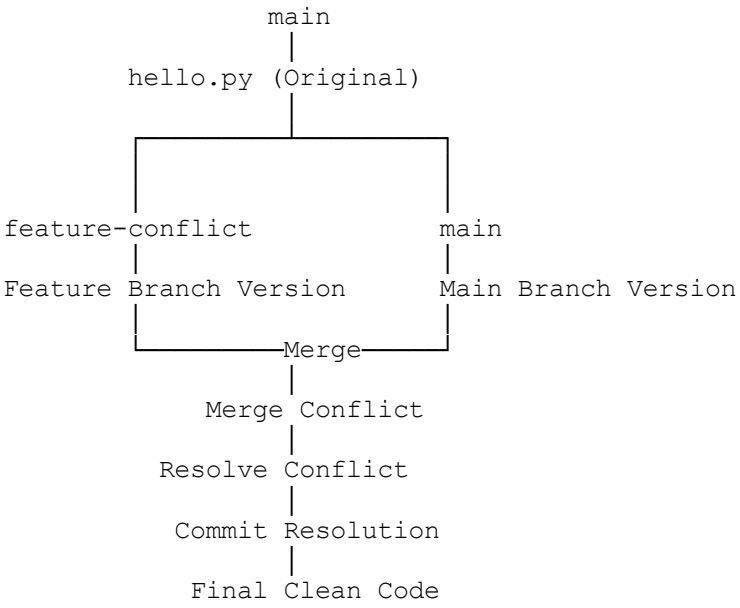
nothing to commit, working tree clean
```

This means the conflict has been resolved successfully.

Final `hello.py`

```
print("Hello Git!")
print("Main Branch Version")
print("Feature Branch Version")
```

Complete Workflow



Commands Summary

Step	Command	Purpose
1	`git checkout -b feature-conflict`	Create a new branch
2	`git add hello.py`	Stage changes
3	`git commit -m "Modified hello.py in feature branch"`	Commit feature changes
4	`git checkout main`	Switch to main
5	`git add hello.py`	Stage main changes
6	`git commit -m "Modified hello.py in main branch"`	Commit main changes
7	`git merge feature-conflict`	Attempt to merge
8	Edit `hello.py`	Resolve conflict manually
9	`git add hello.py`	Stage resolved file
10	`git commit -m "Resolved merge conflict"`	Finish the merge
11	`git status`	Verify repository is clean

Viva Questions

Q1. What is a merge conflict?

Answer: A merge conflict occurs when Git cannot automatically merge changes because the same part of a file has been modified differently in two branches.

Q2. Why does a merge conflict occur?

Answer: It occurs when two branches make different changes to the same line or nearby lines of the same file.

Q3. How do you resolve a merge conflict?

Answer: Open the conflicted file, remove the conflict markers, choose or combine the required changes, save the file, stage it using `git add`, and complete the merge with `git commit`.

Q4. What are conflict markers in Git?

Answer: Git inserts `<<<<<<<`, `=======`, and `>>>>>>>` into the file to show the conflicting changes from each branch.

Q5. Which command is used to merge branches?

Answer: The `git merge <branch-name>` command is used to merge one branch into the current branch.

This practical demonstrates the complete lifecycle of **creating a merge conflict and resolving it**, which is exactly what examiners typically expect in a Git practical.

PRACTICAL 4:
Creating and Managing a Virtual Machine.
a. Set up a virtual machine on a cloud provider or locally (e.g., using VMWare or Azure).
b. Configure networking concepts like virtual networks (Vnet), IP addresses, and ports
c. Deploy the Flask/FastAPI app manually on the VM.

To create and manage a Virtual Machine using VMware, configure networking, and deploy a Flask application manually on Ubuntu.

Software Requirements

- VMware Workstation
- Ubuntu (already installed)
- Python 3
- Flask
- Terminal
- Web Browser

Part (a): Set Up a Virtual Machine Using VMware

Step 1: Open VMware Workstation

1. Launch **VMware Workstation**.
2. Select your **Ubuntu Virtual Machine**.
3. Click **Power On**.

Step 2: Log in to Ubuntu

Enter your Ubuntu username and password.

Step 3: Update Ubuntu

Open the Terminal and run:

```
sudo apt update
sudo apt upgrade -y
```

Step 4: Verify Python Installation

```
python3 --version
```

Example Output:

```
Python 3.12.3
```

If Python is not installed:

```
sudo apt install python3 python3-pip -y
```

Step 5: Install Flask

```
pip3 install flask
```

Verify:

```
pip3 show flask
```

Result (Part a)

The Ubuntu Virtual Machine is successfully created and configured using VMware.

Part (b): Configure Networking (Virtual Network, IP Address, and Ports)

Step 1: Check the IP Address

Run:

```
ip addr
```

or

```
hostname -I
```

Example Output:

```
192.168.43.128
```

This is your VM's IP address.

Step 2: Understand VMware Network Modes

Network Mode	Description
NAT	VM shares the host's internet connection (recommended for beginners).
Bridged	VM gets its own IP address on the same network as the host.
Host-only	Communication only between the host and the VM; no internet access.

Recommended: Use **NAT** unless your assignment specifies otherwise.

To check or change the mode:

1. Shut down the VM.
2. In VMware, select the VM.
3. Go to **VM → Settings → Network Adapter**.
4. Choose **NAT** (or **Bridged**, if required).

Step 3: Verify Internet Connectivity

```
ping google.com
```

If you receive replies, the network is working.

Step 4: Check Listening Ports

```
sudo ss -tuln
```

or

```
netstat -tuln
```

Typical output:

```
tcp LISTEN 0 128 0.0.0.0:5000
```

Port **5000** is commonly used by Flask.

Step 5: Firewall (if enabled)

Allow port **5000**:

```
sudo ufw allow 5000
```

Check the firewall status:

```
sudo ufw status
```

Result (Part b)

The VM network is configured, the IP address is verified, and the required application port is available.

Part (c): Deploy the Flask Application Manually

Step 1: Create a Project Folder

```
mkdir FlaskApp
cd FlaskApp
```

Step 2: Create `app.py`

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Hello from Ubuntu Virtual Machine!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 3: Run the Application

```
python3 app.py
```

Expected output:

```
* Running on http://0.0.0.0:5000
```

Step 4: Test the Application

If you're using **NAT**, first test inside the VM:

Open Firefox in Ubuntu:

```
http://localhost:5000
```

Output:

```
Hello from Ubuntu Virtual Machine!
```

If you're using **Bridged** or have configured **NAT port forwarding**, you can also access it from the host using:

```
http://<VM_IP_Address>:5000
```

Example:

```
http://192.168.43.128:5000
```

Step 5: Stop the Flask Server

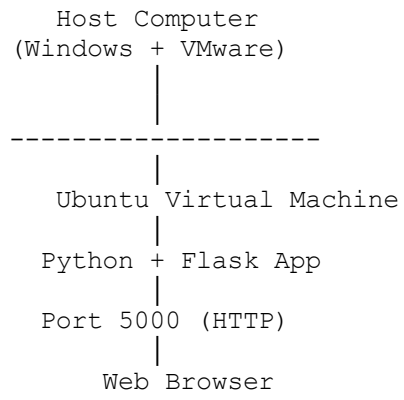
Press:

```
Ctrl + C
```

Project Structure

```
FlaskApp/
└── app.py
```

Architecture



Commands Summary

Command	Purpose
`sudo apt update`	Update package list
`python3 --version`	Check Python version
`pip3 install flask`	Install Flask
`hostname -I`	Display the VM's IP address
`ip addr`	Show network interfaces and IP addresses
`sudo ss -tuln`	Show listening ports
`sudo ufw allow 5000`	Allow Flask through the firewall
`python3 app.py`	Start the Flask application

Practical 5: Dockerize a Flask Application

Aim

To create a Docker image for a Flask application using a Dockerfile.

Objective

- Create a Flask application.
- Create a requirements.txt file.
- Write a Dockerfile.
- Build a Docker image.
- Verify the Docker image.

Software Requirements

Software	Version
Windows 10/11	Any
Python	3.12+
Docker Desktop	Latest
VS Code	Latest
Flask	Latest

A. Write a Dockerfile for your Flask/FastAPI application and build a Docker image

Step 1: Install Docker Desktop

1. Open your browser.
2. Visit **https://www.docker.com/products/docker-desktop/**
3. Download Docker Desktop for Windows.
4. Install it.
5. Restart your computer if prompted.
6. Open Docker Desktop.
7. Wait until the Docker Engine status shows **Running**.

Step 2: Verify Docker Installation

Open **Command Prompt** or **PowerShell**.

Type:

docker --version

Example Output

Docker version 28.3.2, build xxxx

Now check

docker info

If Docker is running, lots of system information will be displayed.

Step 3: Create Project Folder

Open Command Prompt.

mkdir Flask_Project

Go inside the folder.

cd Flask_Project

Step 4: Open the Folder in VS Code

Type

code .

VS Code opens.

Step 5: Create Flask Application

Create a new file

app.py

Write the following code.

from flask import Flask

```
app = Flask(__name__)

@app.route("/")
def home():
    return "Welcome to Dockerized Flask Application"

@app.route("/about")
def about():
    return "Docker Practical"

if __name__=="__main__":
    app.run(host="0.0.0.0",port=5000)
```

Explanation

Statement	Purpose
Flask()	Creates the Flask application
@app.route("/")	Defines the Home page
@app.route("/about")	Defines another page
host="0.0.0.0"	Allows Docker to access the application
port=5000	Flask runs on port 5000

Step 6: Create Virtual Environment (Optional but Recommended)

```
python -m venv venv
```

Activate

Windows

```
venv\Scripts\activate
```

You should see

```
(venv)
```

at the beginning of the command prompt.

Step 7: Install Flask

```
pip install flask
```

Wait until installation completes.

Step 8: Test the Application

Run

python app.py

Output

Running on

http://127.0.0.1:5000

Open browser

http://localhost:5000

Output

Welcome to Dockerized Flask Application

Also check

http://localhost:5000/about

Output

Docker Practical

Step 9: Create requirements.txt

This file contains all Python packages required by the application.

Method 1 (Recommended)

pip freeze > requirements.txt

Open the file.

It may look like

blinker==1.9.0

click==8.2.1
Flask==3.1.0
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.2
Werkzeug==3.1.3

Method 2

Create

requirements.txt

Write

Flask

Method 1: Automatically Create `requirements.txt` (Recommended)

Step 1: Open VS Code

Open your Flask project.

Example:

```
Flask_Project
```

Step 2: Open the Terminal

Click

```
Terminal
↓
New Terminal
```

You should see something like:

```
PS C:\Users\YourName\Documents\Flask_Project>
```

Step 3: Activate the Virtual Environment (if you created one)

If your virtual environment is named **venv**, type:

```
venv\Scripts\activate
```

After activation, your prompt will look similar to:

```
(venv) PS C:\Users\YourName\Documents\Flask_Project>
```

If you did **not** create a virtual environment, you can skip this step.

Step 4: Check That Flask Is Installed

Type:

```
pip list
```

You should see something like:

```
Package      Version
------------ -------
Flask        3.1.0
click        8.2.1
Werkzeug     3.1.3
Jinja2       3.1.6
```

If Flask is **not** listed, install it:

```
pip install flask
```

Step 5: Generate `requirements.txt`

Type:

```
pip freeze > requirements.txt
```

Press **Enter**.

Nothing will appear on the screen, but the file has been created.

Step 6: Verify the File

In VS Code, you should now see:

```
Flask_Project
│
├── app.py
├── requirements.txt
├── Dockerfile
```

Double-click **requirements.txt**.

It may contain:

```
blinker==1.9.0
click==8.2.1
Flask==3.1.0
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.2
Werkzeug==3.1.3
```

This is exactly what Docker will use to install the required packages.

Step 10: Create Dockerfile

Create a file named

Dockerfile

Important: Do **not** give it an extension like .txt. The filename must be exactly Dockerfile.

Paste the following:

```
# Step 1: Use Python base image
FROM python:3.12-slim

# Step 2: Set working directory
WORKDIR /app

# Step 3: Copy requirements file
COPY requirements.txt .

# Step 4: Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Step 5: Copy all project files
COPY . .

# Step 6: Open Flask port
EXPOSE 5000

# Step 7: Start the application
CMD ["python","app.py"]
```

Understanding Each Dockerfile Command

1. FROM

FROM python:3.12-slim

Purpose

- Downloads the official Python image.
- This becomes the base operating system for your container.

---

2. WORKDIR

WORKDIR /app

Purpose

Creates a folder

/app

inside Docker.

All files are copied here.

---

3. COPY

COPY requirements.txt .

Purpose

Copies

requirements.txt

from your computer into the Docker container.

---

4. RUN

RUN pip install --no-cache-dir -r requirements.txt

Purpose

Installs Flask and any other required Python packages inside the Docker image.

---

5. COPY

COPY . .

Purpose

Copies your project files (app.py, etc.) into the container.

---

6. EXPOSE

EXPOSE 5000

Purpose

Tells Docker that the application listens on port **5000**.

---

7. CMD

CMD ["python","app.py"]

Purpose

Specifies the command Docker runs automatically when the container starts.

Step 11: Verify Project Structure

Your folder should look like this:

```
Flask_Project/
│
├── app.py
├── Dockerfile
├── requirements.txt
└── venv/      (optional)
```

Step 12: Open Terminal

Make sure you are inside

Flask_Project

Verify by running:

dir

You should see

app.py

Dockerfile

requirements.txt

Step 13: Build Docker Image

Run

docker build -t flask-app .

Explanation

Part	Meaning
docker	Docker command
build	Build an image
-t	Assign a tag (name)
flask-app	Name of the image
.	Current directory contains the Dockerfile

Docker will:

1. Download the Python image (if not already available).
2. Copy your files.
3. Install Flask.
4. Create the Docker image.

Example output:

Successfully built 8b1234567890

Successfully tagged flask-app:latest

Step 14: Verify the Docker Image

Run:

docker images

Example output:

Repository	Tag	Image ID	Size
flask-app	latest	8b1234567890	170 MB

This confirms that the image has been built successfully.

Expected Result

- Docker Desktop is installed and running.
- Flask application works locally.
- requirements.txt is created.
- Dockerfile is created.
- Docker image is successfully built.
- The image appears in the output of docker images.

Common Errors and Solutions

Error	Possible Cause	Solution
docker: command not found	Docker Desktop is not installed or not running	Install Docker Desktop and ensure it is running
Dockerfile not found	Incorrect filename or wrong folder	Name the file exactly Dockerfile and run the command from the project folder
No module named flask	Flask not installed or missing from requirements.txt	Install Flask (pip install flask) and regenerate requirements.txt
Build fails while installing packages	Incorrect package name or internet issue	Check requirements.txt and try the build again

This completes **Part (a): Write a Dockerfile for your Flask/FastAPI application and build a Docker image**. The next practical would be **(b) Run the Docker container and test the application**, followed by **(c) Push the Docker image to Docker Hub**.

b. Run the Docker Container and Test the Flask Application

Aim

To run the Docker container for the Flask application and verify that the application is working correctly.

Prerequisites

Before performing this practical, ensure that:

- Docker Desktop is installed and running.
- The Flask application is created.
- The Dockerfile is created.
- The Docker image has been built successfully.

Step 1: Open Docker Desktop

1. Click **Start**.
2. Search for **Docker Desktop**.
3. Open Docker Desktop.
4. Wait until the status shows:

Engine Running

Step 2: Open the Project Folder

Open **Command Prompt**, **PowerShell**, or the **VS Code Terminal**.

Navigate to your project folder:

cd C:\Users\YourName\Documents\Flask_Project

Replace the path with your actual project folder.

Step 3: Verify the Docker Image

Check whether the Docker image exists.

docker images

Example Output:

```
REPOSITORY      TAG       IMAGE ID       CREATED         SIZE
flask-app       latest    a1b2c3d4e5f6   5 minutes ago   170MB
```

If you do not see flask-app, build the image first:

docker build -t flask-app .

Step 4: Run the Docker Container

Run the Flask container:

docker run -d -p 5000:5000 flask-app

Explanation

Command	Description
docker run	Creates and starts a new container
-d	Runs the container in detached (background) mode
-p 5000:5000	Maps port 5000 on your computer to port 5000 inside the container
flask-app	Name of the Docker image

Step 5: Verify the Container

Check whether the container is running:

docker ps

Example Output:

```
CONTAINER ID   IMAGE       COMMAND           STATUS          PORTS
4a2d6c8f1b9e   flask-app   "python app.py"   Up 20 seconds   0.0.0.0:5000->5000/tcp
```

If the status is **Up**, the application is running successfully.

Step 6: Test the Application

Open any web browser.

Enter the following URL:

http://localhost:5000

Expected Output

Welcome to Dockerized Flask Application

If your application has another route such as /about, test it as well:

http://localhost:5000/about

Expected Output:

Docker Practical

Step 7: Test Using the Terminal (Optional)

Instead of using a browser, you can also test the application with:

curl http://localhost:5000

Expected Output:

Welcome to Dockerized Flask Application

Step 8: Check Container Logs

If the browser does not display the application, view the logs:

docker logs <container_id>

Example:

docker logs 4a2d6c8f1b9e

Example Output:

* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000

The logs help identify startup errors or missing dependencies.

Step 9: Stop the Container

First, list the running containers:

docker ps

Then stop the container:

docker stop <container_id>

Example:

docker stop 4a2d6c8f1b9e

Step 10: Restart the Container

List all containers:

docker ps -a

Restart the stopped container:

docker start <container_id>

Example:

docker start 4a2d6c8f1b9e

Open the browser again:

http://localhost:5000

The application should be accessible again.

Expected Output

- The Docker container starts successfully.
- The container appears in the output of docker ps.
- The Flask application opens successfully at http://localhost:5000.
- The application responds correctly to all defined routes.

Common Errors and Solutions

Error	Cause	Solution
Unable to find image 'flask-app:latest' locally	Docker image has not been built	Run docker build -t flask-app .
Bind for 0.0.0.0:5000 failed: port is already allocated	Port 5000 is already in use	Stop the existing container (docker stop <container_id>) or use another port, e.g. docker run -d -p 5001:5000 flask-app
Browser shows **Problem loading page**	Container is not running or the application crashed	Check docker ps and then docker logs <container_id>
No such container	Incorrect container ID	Copy the correct ID from docker ps or docker ps -a

---

Practical 5 c

Step 1: Open Command Prompt or VS Code Terminal

Run:

```
docker images
```

You should see something like:

```
REPOSITORY      TAG       IMAGE ID
flask-app       latest    a1b2c3d4e5f6
```

If you don't see `flask-app`, you need to build the image first.

Step 2: Log in to Docker Hub

Run:

```
docker login
```

Enter:

- **Username:** Your Docker Hub username
- **Password:** Your Docker Hub password (or Personal Access Token if your account requires one)

If successful, you'll see:

```
Login Succeeded
```

Step 3: Check Which Account Is Logged In

Run:

```
docker info
```

Look for:

```
Username: yourusername
```

Make sure it matches your Docker Hub account.

Step 4: Tag Your Image

Suppose your Docker Hub username is:

`nikhita123`

Run:

```
docker tag flask-app nikhita123/flask-app:latest
```

Replace `nikhita123` with **your own Docker Hub username**.

Step 5: Verify the Tag

Run:

```
docker images
```

You should now see something like:

```
REPOSITORY                TAG
flask-app                 latest
nikhita123/flask-app      latest
```

Step 6: Push the Image

Run:

```
docker push nikhita123/flask-app:latest
```

Replace `nikhita123` with your actual Docker Hub username.

If everything is correct, Docker will upload the image.

Step 7: Verify on Docker Hub

1. Open **https://hub.docker.com/**
2. Log in.
3. Click **Repositories**.

4. You should see **flask-app** listed.